

Spin BuAli — Function & Class Reference

A complete inventory of every function and class, organized by folder and file. Written as a code-reading aid, not user documentation.

controller/

main.py

FastAPI app (`app`), global `SESSION`

- `_redact_session(session)` — strips API keys out of a session dict before returning it to a client
- `health()` — `GET /` — reports STT/LLM reachability and whether default API keys are configured
- `list_models()` — `GET /models` — proxies STT's available model list
- `list_languages()` — `GET /languages` — proxies STT's supported languages
- `status()` — `GET /status` — returns the redacted active session, or `{"active": false}`
- `start_session(req)` — `POST /session` — validates pipeline name, slot count, and reachability of whatever servers the chosen pipeline needs; stores the session
- `run(file, language, stt_api_key, stt_base_url, llm_api_key, llm_base_url, stt_slots_json)` — `POST /run` — reads the uploaded audio, dispatches to whichever pipeline function matches `SESSION.pipeline`
- `unload()` — `POST /session/unload` — tells STT and Core_LLM to unload their models, clears `SESSION`

pipelines.py

No classes, all functions

- `_run_stt_slots(audio, stt_slots, language)` — runs every configured STT slot, returns `{"transcript_1":..., ...}`
- `_reconcile(outputs, llm_model, llm_api_key, llm_base_url)` — sends all produced transcripts to the LLM in one call to merge/clean them into the report template
- `run_separate(...)` — the "Separate" pipeline: `_run_stt_slots` then `_reconcile`
- `_transcribe_and_report_directly(audio, audio_filename, reference_outputs, ...)` — sends raw audio (plus any reference transcripts) straight to an audio-capable LLM
- `run_multimodal(...)` — the "Multimodal" pipeline: calls `_transcribe_and_report_directly` with no reference transcripts
- `run_hybrid(...)` — the "Hybrid" pipeline: `_run_stt_slots` first, then `_transcribe_and_report_directly` with those as reference

config.py

No functions/classes — module-level constants loaded from `.env`: `STT_URL`, `LLM_URL`, `HOST`, `PORT`, `HTTP_TIMEOUT`, `OPENAI_API_KEY` / `OPENAI_BASE_URL` / `OPENAI_STT_MODEL`, `API_PREFIX`, `GEMINI_API_KEY` / `GEMINI_BASE_URL` / `GEMINI_PREFIX`, `MAX_STT_SLOTS`.

schemas.py

Pydantic models, no free functions

- `SttSlotConfig` — one STT engine's config (model/api_key/base_url/language)
- `SessionRequest` — the `POST /session` request body
- `Session` — the stored session state (adds `stt_ready` / `llm_ready`)

stt_llm_client.py

All functions — the HTTP client layer to STT/Core_LLM and external APIs

- `is_api_model(model)` / `strip_api_prefix(model)` — detect/strip the `"openai:"` prefix
- `is_gemini_model(model)` / `strip_gemini_prefix(model)` — detect/strip the `"gemini:"` prefix
- `is_cloud_model(model)` — true if either of the above
- `_client()` — builds an `httpx.Client` with the configured timeout
- `_resolve_api_key(api_key)` / `_resolve_base_url(base_url)` — fall back to config defaults, error if no OpenAI key available
- `stt_health()` — pings the local STT service
- `stt_models()` — fetches STT's model list
- `stt_load(model)` — tells STT to load a model
- `stt_transcribe(audio, filename, language)` — calls local STT's `/transcribe`
- `stt_api_transcribe(...)` — calls an external OpenAI-compatible STT API
- `stt_languages()` — fetches STT's supported languages
- `stt_unload()` — tells STT to unload
- `llm_health()` — pings local Core_LLM
- `llm_chat(messages, model, response_format)` — calls local Core_LLM's `/chat`
- `llm_api_chat(...)` — calls an external OpenAI-compatible `/chat/completions`
- `audio_format_from_filename(filename, model)` — validates the audio container format against whichever provider's restrictions apply
- `llm_api_chat_audio(...)` — OpenAI-compatible audio-input chat (base64-encodes the audio into the message)
- `_resolve_gemini_api_key(api_key)` / `_resolve_gemini_base_url(base_url)` — same fallback pattern for Gemini
- `llm_gemini_chat_audio(...)` — calls Gemini's native `generateContent` API directly with inline audio data
- `llm_local_chat_audio(...)` — calls local Core_LLM's `/chat_audio`
- `llm_chat_audio(...)` — dispatcher: routes to Gemini, OpenAI-compatible, or local based on the model string

- `chat(messages, model, response_format, api_key, base_url)` — dispatcher for text-only chat: local vs. OpenAI-compatible
- `llm_unload(model)` — tells Core_LLM to unload (no-op for cloud models)
- `transcribe_slot(audio, slot, default_language)` — runs one STT slot, loading the model first if it's local
- `extract_json(text)` — pulls a JSON object out of an LLM reply that may be wrapped in prose/code fences

stt/

app/main.py

FastAPI app (`app`), global `manager`

- `lifespan(app)` — optionally preloads a default model at startup
- `_read_audio(raw)` — decodes uploaded bytes into a mono float32 array + sample rate
- `list_models()` — GET `/models`
- `load_model(key)` — POST `/models/{key}/load`
- `unload_model()` — POST `/models/unload`
- `list_languages()` — GET `/languages`
- `transcribe(file, language)` — POST `/transcribe`

app/model.py

- `resample(audio, sr, target_sr)` — resamples audio via torchaudio if needed
- `BaseSTTModel` (ABC) — `__init__`, abstract `load()`, abstract `transcribe()`, `unload()`
 - `WhisperModel(BaseSTTModel)` — `load()`, `transcribe()` (uses `forced_decoder_ids` for language)
 - `SeamlessV2Model(BaseSTTModel)` — `__init__`, `load()`, `transcribe()`
 - `SeamlessV1Model(BaseSTTModel)` — `__init__`, `load()`, `transcribe()` (different output decoding than v2)
 - `CTCModel(BaseSTTModel)` — `load()`, `transcribe()` (plain wav2vec2)
 - `MMSModel(CTCModel)` — `__init__`, `load()` (loads a language adapter on top of the shared backbone)
- `build_model(key)` — factory: instantiates (doesn't load) the class registered for `key`
- `ModelManager` — `__init__`, `available()`, `loaded` (property), `load(key)`, `transcribe(audio, sr, language)`, `unload()`

app/config.py

Constants only: `DEVICE`, `HOST`, `PORT`, `ALLOWED_ORIGINS`, `TARGET_SAMPLE_RATE`, `SUPPORTED_LANGUAGES`, `DEFAULT_LANGUAGE`, `PRELOAD_MODEL`, `MODEL_REGISTRY` (the 10 STT models, ranked by clinic-noise WER).

app/schemas.py

Pydantic models only: `ModelInfo`, `LanguagesResponse`, `LoadResponse`, `TranscriptionResult`.

app/__init__.py

Empty — just a docstring marking the package.

tests/test_api.py

- `test_list_models_reports_registry()` — checks `/models` lists the registry
- `test_unknown_model_is_rejected()` — checks a bad model key 404s
- `test_transcribe_requires_a_loaded_model()` — checks `/transcribe` 409s with nothing loaded

core_llm/

main.py

FastAPI app (`app`)

- `health()` — GET `/`
- `list_models()` — GET `/models`
- `chat(req)` — POST `/chat`
- `unload(model)` — POST `/unload`
- `chat_audio_models()` — GET `/chat_audio/models`
- `chat_audio(file, system_prompt, text, model, temperature)` — POST `/chat_audio`
- `chat_audio_unload()` — POST `/chat_audio/unload` (alias of `/unload`)

model.py

- `_generation_kwargs(temperature)` — greedy vs. sampled decoding based on temperature
- `BaseLLM` (ABC) — `__init__` , abstract `load()` , abstract `chat()` , `unload()`
 - `TextOnlyModel(BaseLLM)` — `load()` , `chat()` (Aya Expanse, Gemma 4 31B — no audio)
- `_last_user_index(messages)` — finds the index of the last user turn (to attach audio to it)
 - `GemmaAudioModel(BaseLLM)` — `load()` , `chat()` (Gemma 4 E4B/12B, audio via `"url"` content key)
 - `QwenOmniModel(BaseLLM)` — `load()` , `chat()` (Qwen3-Omni, audio via `"path"` content key)
- `LLMManager` — `__init__` , `available(audio_only)` , `loaded` (property), `_ensure_loaded(key)` , `chat(key, messages, audio, audio_format, temperature, response_format)` , `unload()`

config.py

Constants only: `HOST` , `PORT` , `ALLOWED_ORIGINS` , model IDs for Aya/Gemma/Qwen, `DEFAULT_MODEL` , `MAX_NEW_TOKENS` , `DEVICE_MAP` .

schemas.py

Pydantic models only: `ChatMessage` , `ChatRequest` , `ChatResponse` , `HealthResponse` .

chat.py

Standalone terminal test script, not used by the API

- `main()` — a manual REPL loop calling `MANAGER.chat` directly, no HTTP

demo_app/

app.py

- `error_detail(exc)` — extracts a readable error message from a failed `requests` call
- `run_bg(fn, *args, **kwargs)` — runs a function in a daemon thread (keeps the UI responsive)
- `ModeSelector(ttk.Frame)` — `__init__`, `is_cloud()` — local-vs-cloud dropdown, reused for every model choice
- `CloudFieldsFrame(ttk.Frame)` — `__init__` — model/API-key/base-URL entry fields
- `SttSlotWidget(ttk.Frame)` — `__init__`, `refresh_local_models()`, `_refresh_clicked()`, `_update_visibility()`, `is_cloud()`, `effective_model()`, `as_slot_config()` — one configurable STT engine slot (up to 3 on screen)
- `MicRecorder` — `__init__`, `_callback(...)`, `start()`, `stop_and_save()` — records mic audio to a temp `.wav`
- `RecordButton(ttk.Button)` — `__init__`, `_toggle()`, `_start()`, `_stop()`
- `ConnectionBar(ttk.Frame)` — `__init__`, `base_url` (property), `check()`, `_check_bg()`, `_set(text, color)` — host/port fields + a live "connected/unreachable" indicator
- `_looks_rtl(text)` — checks if a line contains Persian/Arabic-range codepoints
- `sanitize_filename_part(name)` — strips filesystem-unsafe characters from a model name for use in a filename
- `build_transcript_filename(audio_path, model_parts, ext)` — builds `<audiofile>_<model>.docx`
- `save_text_as_docx(text, path)` — writes each line as a paragraph, RTL-aligned if Persian
- `TranscriptSaver(ttk.Frame)` — `__init__`, `note_audio_path(...)`, `_browse()`, `_save()` — the "Save Transcript" widget
- `OutputBox(scrolledtext.ScrolledText)` — `__init__`, `write(text)` — read-only result display
- `ScrollableFrame(ttk.Frame)` — `__init__` — canvas+scrollbar wrapper so content isn't clipped by window size
- `App(tk.Tk)` — the main window:
 - `__init__` — builds the whole UI (connection bar, pipeline dropdown, 3 STT slot widgets, LLM section, session buttons, audio input, run button, output box, transcript saver)
 - `_pipeline_value()` — maps the dropdown label to `"separate"/"multimodal"/"hybrid"`
 - `_update_pipeline_visibility()` — shows/hides the STT-slots panel depending on pipeline
 - `_update_llm_section()` — filters the local-model dropdown to audio-capable models when needed; shows the Gemini hint for cloud audio modes
 - `_get_local_stt_models()` — returns the cached STT model list

- `refresh_stt_models()` / `_refresh_stt_models_bg()` / `_apply_stt_models()` — fetch STT's model list in the background and push it into the slot widgets
- `_effective_llm_model()` — resolves the actual model string to send (local key, or "openai:" / "gemini:" prefixed)
- `start_session()` / `_start_session_bg(payload)` — `POST /session` in the background
- `unload_session()` / `_unload_session_bg()` — `POST /session/unload`
- `browse()` — file picker for an audio file
- `run()` / `_run_bg(path, llm_key, llm_base)` — `POST /run` with the audio file, writes the JSON result to the output box
- `_get_text_and_parts()` — extracts the final text + model name for the transcript saver